



**UNIVERSIDADE FEDERAL DO MARANHÃO
BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA
(BICT)**

RELATÓRIO DE PLANEJAMENTO E CRONOGRAMA

**SISTEMA DE ESTACIONAMENTO
INTELIGENTE**

Docente: Luiz Henrique Neves Rodrigues

Discentes:

Ruan Pactrick de Sousa e Sousa
Leonardo Sampaio Serra
João Pedro Moura de Mendonça
Marcos Antonio Nunes de Alencar
Deyvison Sousa Nogueira

SÃO LUÍS (MA) 2026

Sumário

Sumário	2
1 APRESENTAÇÃO	3
2 RESUMO DO QUE FOI FEITO	3
3 VISÃO GERAL DO SISTEMA	4
4 DEFINIÇÃO DE VARIÁVEIS	4
5 TABELA VERDADE	5
6 MAPA DE KARNAUGH E EXPRESSÕES SIMPLIFICADAS	6
6.1 Expressão para Contagem de Vagas - CV	6
6.2 Expressão para Liberação de Vaga - LV	6
6.3 Expressão para o LED Vermelho - LedR	6
6.4 LEDs Vermelhos de Vaga	6
7 PROTÓTIPO NO TINKERCAD E VISTA ESQUEMÁTICA	6
7.1 Vista Esquemática do Projeto	7
7.1.1 Análise da Vista Esquemática (Parte 1)	9
7.1.2 Análise da Vista Esquemática (Parte 2)	10
8 CÓDIGOS FONTE (ARDUINO)	10
8.1 Código do Arduino Gestor	10
8.1.1 Análise Lógica do Arduino Gestor	12
8.2 Código do Arduino Porteiro	13
8.2.1 Análise Lógica do Arduino Porteiro	15
9 DISTRIBUIÇÃO GERAL DAS RESPONSABILIDADES	16
10 CONSIDERAÇÕES FINAIS	16
REFERÊNCIAS	17

1 APRESENTAÇÃO

O Sistema de Estacionamento Inteligente propõe uma solução simples e eficaz para esse cenário, utilizando circuitos digitais, sensores ultrassônicos e sinalização visual automatizada. O projeto foi desenvolvido como protótipo simulado na plataforma Tinkercad, com foco didático no contexto da disciplina de Circuitos Digitais.

O sistema monitora entradas e saídas de veículos em tempo real, exibe o número de vagas disponíveis em um display de 7 segmentos e sinaliza o estado do estacionamento por LEDs, tudo de forma autônoma, sem intervenção humana.

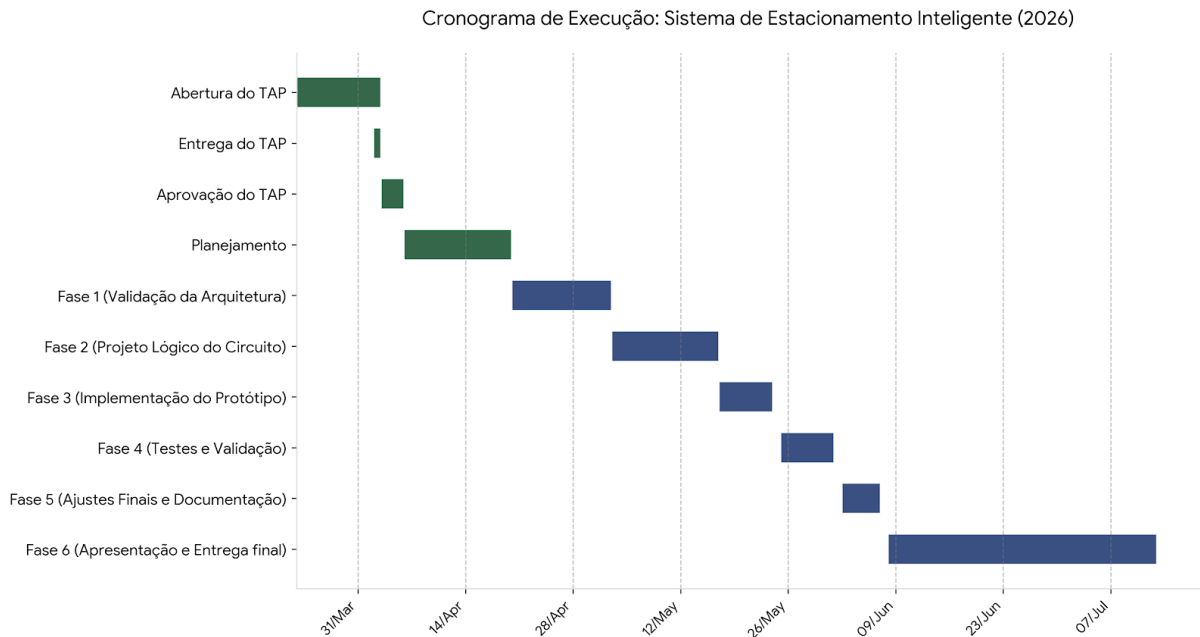
2 RESUMO DO QUE FOI FEITO

O projeto já percorreu todas as fases do cronograma:

- **Requisitos definidos:** regras de contagem bidirecional, limite de 9 vagas, lógica de lotação e bloqueio automático.
- **Projeto lógico concluído:** definição das variáveis de entrada/saída, elaboração da tabela verdade, simplificação por Mapa de Karnaugh e obtenção das expressões booleanas finais.
- **Protótipo implementado:** montagem completa no Tinkercad com sensores, Arduino Uno, 9 LEDs de vaga, LEDs vermelho de estado e display I²C. O código Arduino foi escrito e validado com simulação funcional.

A Figura 1 ilustra o cronograma detalhado do projeto, destacando as fases já concluídas e as etapas em andamento até a entrega final.

Figura 1 – Diagrama de Gantt - Cronograma Atualizado do Projeto.



Fonte: Elaborada pelos autores (2026).

3 VISÃO GERAL DO SISTEMA

O sistema opera de forma completamente autônoma com os seguintes componentes:

Tabela 1 – Componentes do Sistema

Componente	Função
2 Sensores HC-SR04	Detectam entrada e saída de veículos.
2 Servos motores	simulam as cancelas/portas.
2 Arduinos Uno	Alimenta e administra todo o circuito.
1 Display I2C	Exibe o número de vagas disponíveis.
2 LEDs Vermelhos e 2 verde	Sinaliza o estado das entradas do estacionamento.
9 LEDs Vermelhos	Indicam visualmente cada vaga ocupada.
13 Resistores	limitar a corrente elétrica.

Fonte: Elaborada pelos autores (2026).

O escopo do protótipo inclui contagem bidirecional de 0 a 9 vagas, sinalização visual por LEDs vermelhos e display, e simulação dos sensores.

4 DEFINIÇÃO DE VARIÁVEIS

As entradas do sistema responsáveis pela detecção dos veículos são:

Tabela 2 – Variáveis de Entrada

Tipo	Símbolo	Descrição
Entrada	SE	Sensor de entrada
Entrada	SS	Sensor de saída
Entrada	L	Lotação

Fonte: Elaborada pelos autores (2026).

As saídas do sistema para a atualização visual do protótipo são:

Tabela 3 – Variáveis de Saída

Tipo	Símbolo	Descrição
Saída	CV	Conta vagas
Saída	LV	Libera vaga
Saída	LedR	LED vermelho de estado
Saída	LEDs Vagas	Conjunto de 9 LEDs vermelhos

Fonte: Elaborada pelos autores (2026).

5 TABELA VERDADE

A tabela verdade foi ajustada considerando a nova lógica do circuito:

Tabela 4 – Tabela Verdade do Sistema

SE	SS	L	CV	LV	LedR	Situação
0	0	0	0	0	0	Nenhum veículo detectado, há vagas
0	0	1	0	0	1	Estacionamento lotado
0	1	0	1	1	0	Veículo saindo e vaga liberada
0	1	1	1	1	0	Veículo saindo mesmo com lotação
1	0	0	1	0	1	Veículo entrando e vaga ocupada
1	0	1	0	0	X	Entrada bloqueada (lotado)
1	1	0	0	0	X	Condição inválida ou instável
1	1	1	0	0	X	Condição inválida ou instável

Fonte: Elaborada pelos autores (2026).

6 MAPA DE KARNAUGH E EXPRESSÕES SIMPLIFICADAS

6.1 Expressão para Contagem de Vagas - CV

O contador será alterado quando o sensor de entrada detectar um veículo (sem lotação) ou quando detectar saída:

$$CV = (SE \cdot L') + SS \quad (1)$$

6.2 Expressão para Liberação de Vaga - LV

A liberação ocorre quando o sensor de saída é ativado:

$$LV = SS \quad (2)$$

6.3 Expressão para o LED Vermelho - LedR

Acende quando há entrada de veículo ou o estacionamento está lotado:

$$LedR = SE + L \quad (3)$$

6.4 LEDs Vermelhos de Vaga

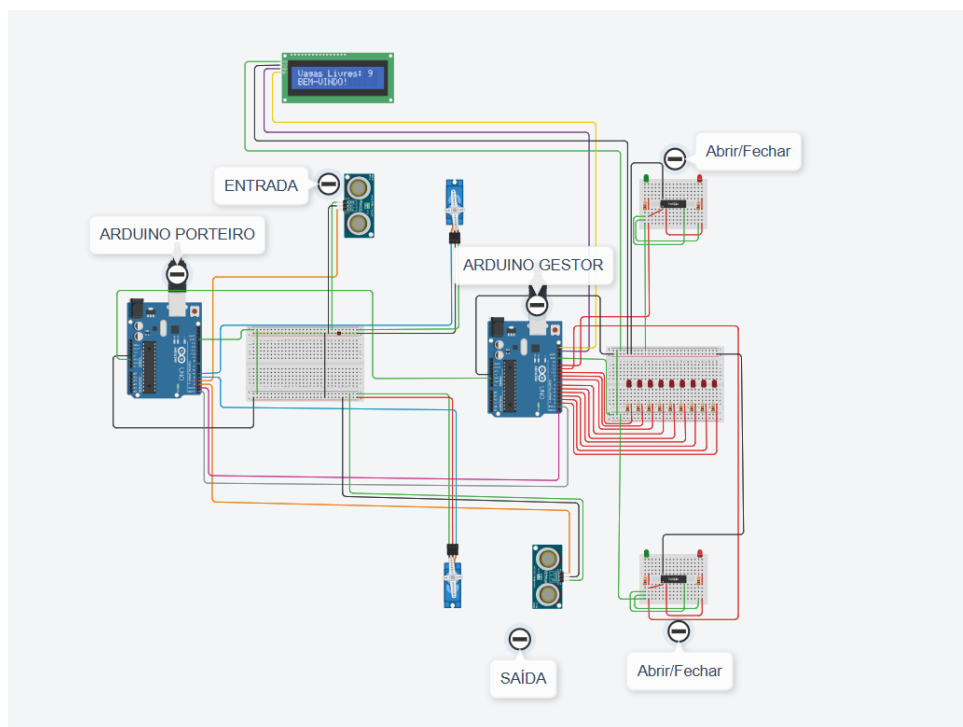
Os nove LEDs não dependem de uma tabela simples, mas da lógica de ocupação:

$$\text{Vagas ocupadas} = \text{MAX_VAGAS} - \text{vagas disponíveis} \quad (4)$$

7 PROTÓTIPO NO TINKERCAD E VISTA ESQUEMÁTICA

O protótipo foi montado na plataforma Tinkercad, reunindo sensores ultrassônicos, Arduino Uno, 9 LEDs de vaga, LEDs de estado e um display LCD I2C. A arquitetura foi dividida entre o **Arduino Gestor** e o **Arduino Porteiro**.

Figura 2 – Visão geral do circuito criado no Tinkercad.

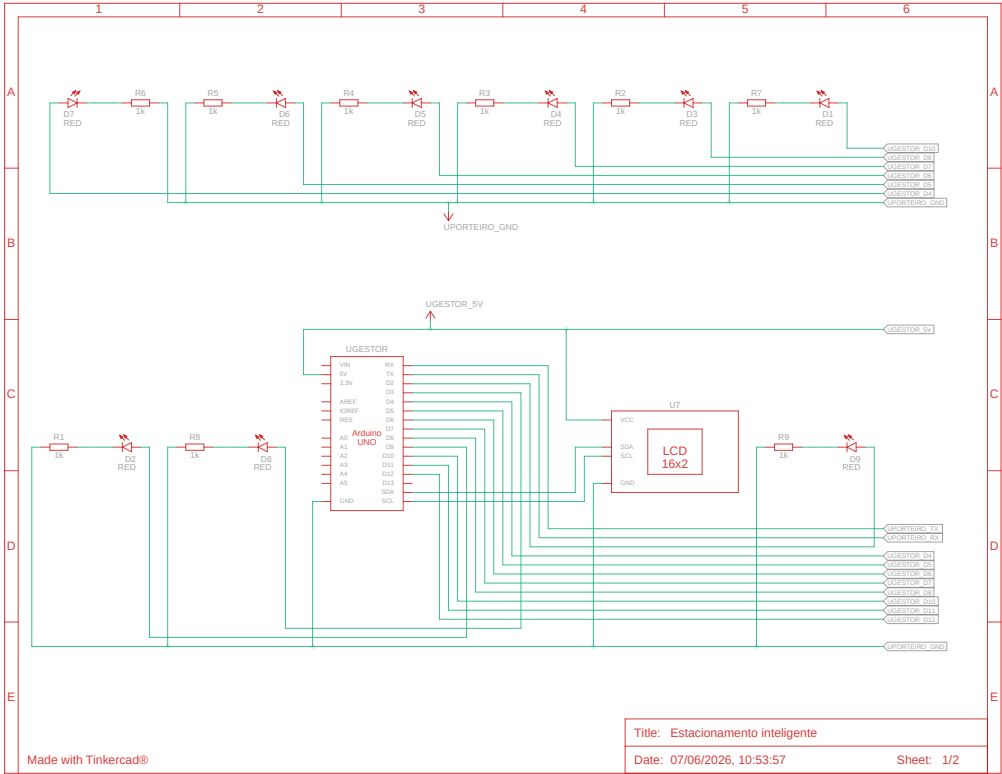


Fonte: Elaborada pelos autores (2026).

7.1 Vista Esquemática do Projeto

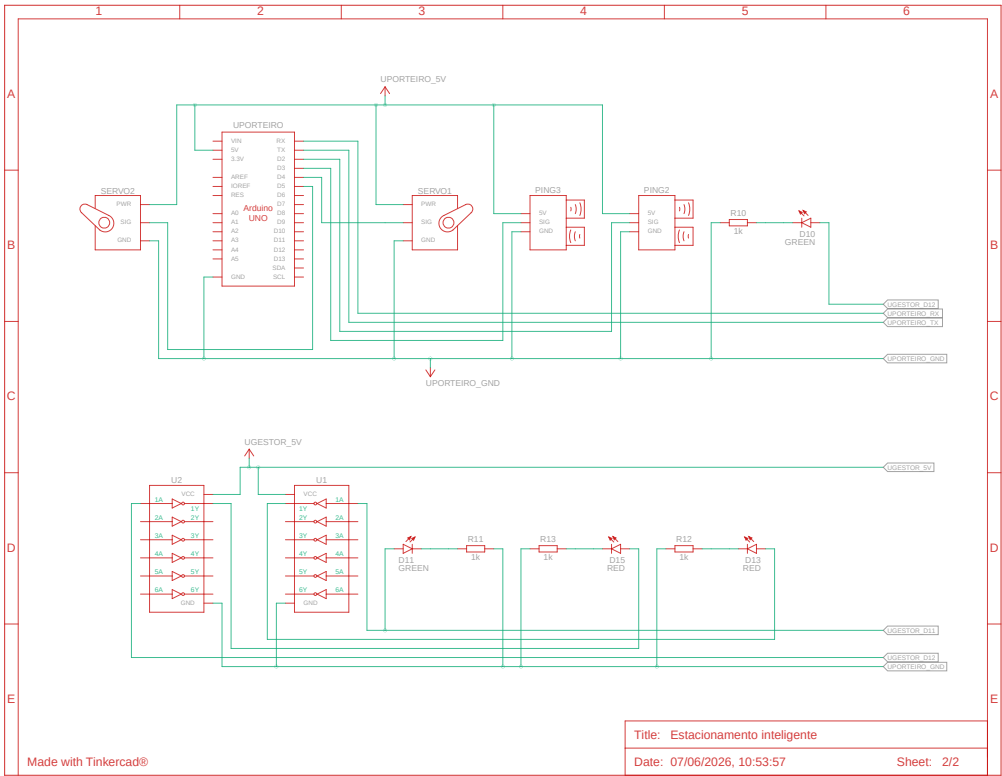
Abaixo estão os diagramas esquemáticos gerados diretamente da plataforma Tinkercad, detalhando as conexões elétricas e lógicas entre os microcontroladores, sensores e atuadores do projeto.

Figura 3 – Vista Esquemática - Diagrama de Conexões (Parte 1).



Fonte: Elaborada pelos autores (2026).

Figura 4 – Vista Esquemática - Diagrama de Conexões (Parte 2).



Fonte: Elaborada pelos autores (2026).

7.1.1 Análise da Vista Esquemática (Parte 1)

A primeira parte do diagrama esquemático concentra-se nos componentes de sinalização visual gerenciados pelo **Arduino Gestor**. Observam-se os seguintes detalhes:

- **Conjunto de LEDs das Vagas:** Estão representados nove LEDs vermelhos (D1 a D9). Cada um possui um resistor limitador de corrente (R2 a R8 e outros não nomeados na imagem, tipicamente de $1k\Omega$) ligado em série para proteger o componente. Estes LEDs estão conectados aos pinos digitais do Arduino Gestor e servem para indicar a ocupação individual de cada uma das 9 vagas.
- **Display LCD I²C:** Ao centro da imagem, o módulo LCD 16x2 é exibido. Este componente recebe alimentação (5V e GND) e comunica-se com o Arduino Gestor através dos pinos SDA (dados) e SCL (clock), característicos do protocolo I²C. A sua função é exibir numericamente as vagas livres e mensagens de estado.
- **Alimentação:** É possível notar as indicações de barramentos de alimentação como UGESTOR_5V e UPORTEIRO_GND, demonstrando que o circuito possui pontos

de interligação de terra (GND) em comum, uma prática fundamental quando se trabalha com múltiplos microcontroladores para garantir uma referência de tensão unificada.

7.1.2 Análise da Vista Esquemática (Parte 2)

A segunda parte do diagrama esquemático ilustra os componentes de detecção, controle físico e lógica adicional, majoritariamente atrelados ao **Arduino Porteiro**.

- **Arduino Porteiro:** Este microcontrolador (representado pelo bloco Arduino UNO à esquerda) é o cérebro das cancelas.
- **Sensores e Atuadores:**
 - **Sensores Ultrassônicos:** Os blocos nomeados como PING1 e PING2 representam os sensores HC-SR04 de entrada e saída. Eles emitem e recebem pulsos sonoros para detectar a presença de veículos.
 - **Servomotores:** Os componentes SERV01 e SERV02 atuam como as cancelas físicas do estacionamento. Eles recebem um sinal PWM do Arduino Porteiro para abrir (movimento de 90°) e fechar (retorno a 0°).
- **Circuitos Lógicos Auxiliares (Sinalização das Portas):** A porção inferior do diagrama apresenta portas inversoras (circuitos integrados como U1 e U2, tipicamente da família 74HC04). Esses componentes lógicos recebem o sinal dos pinos de entrada/saída e controlam LEDs de status (como os LEDs Verde D10, D11 e Vermelhos D12, D13). O sinal que comanda o servomotor para abrir a cancela passa por um desses inversores para garantir que o LED vermelho apague e o verde acenda simultaneamente e de forma automatizada por hardware, desonerando o código do microcontrolador.

8 CÓDIGOS FONTE (ARDUINO)

Para otimizar o processamento e separar as responsabilidades, o sistema utiliza dois microcontroladores.

8.1 Código do Arduino Gestor

Responsável pelo gerenciamento do display e dos LEDs indicadores de ocupação das vagas.

```
1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3
```

```

4 LiquidCrystal_I2C lcd(0x27, 16, 2);
5
6 int vagasOcupadas = 0;
7
8 // NOVOS PINOS DOS LEDS DAS PORTAS
9 int pinoLedEntrada = 12;
10 int pinoLedSaida = 11;
11
12 void setup() {
13     Serial.begin(9600);
14
15     // Configura os leds das vagas (2 a 10)
16     for (int pino = 2; pino <= 10; pino++) {
17         pinMode(pino, OUTPUT);
18         digitalWrite(pino, LOW);
19     }
20
21     // Configura os novos LEDs indicadores das portas
22     pinMode(pinoLedEntrada, OUTPUT);
23     pinMode(pinoLedSaida, OUTPUT);
24     digitalWrite(pinoLedEntrada, LOW);
25     digitalWrite(pinoLedSaida, LOW);
26
27     lcd.init();
28     lcd.backlight();
29     atualizarEcra();
30 }
31
32 void loop() {
33     if (Serial.available() > 0) {
34         char mensagem = Serial.read();
35
36         // --- Comandos da Entrada ---
37         if (mensagem == 'E') {
38             digitalWrite(pinoLedEntrada, HIGH); // Acende o LED indicador
39             if (vagasOcupadas < 9) {
40                 vagasOcupadas++;
41                 atualizarLuzesEEcra();
42             }
43         }
44         else if (mensagem == 'e') {
45             digitalWrite(pinoLedEntrada, LOW); // Apaga o LED indicador
46         }
47
48         // --- Comandos da Saída ---
49         if (mensagem == 'S') {
50             digitalWrite(pinoLedSaida, HIGH); // Acende o LED indicador

```

```

51     if (vagasOcupadas > 0) {
52         vagasOcupadas--;
53         atualizarLuzesEEcra();
54     }
55 }
56 else if (mensagem == 's') {
57     digitalWrite(pinoLedSaida, LOW); // Apaga o LED indicador
58 }
59 }
60 }
61
62 void atualizarLuzesEEcra() {
63     for (int pino = 2; pino <= 10; pino++) {
64         int numeroDoLed = pino - 1;
65         if (numeroDoLed <= vagasOcupadas) {
66             digitalWrite(pino, HIGH);
67         } else {
68             digitalWrite(pino, LOW);
69         }
70     }
71     atualizarEcra();
72 }
73
74 void atualizarEcra() {
75     int vagasLivres = 9 - vagasOcupadas;
76
77     lcd.clear();
78     lcd.setCursor(0, 0);
79     lcd.print("Vagas Livres: ");
80     lcd.print(vagasLivres);
81
82     lcd.setCursor(0, 1);
83     if (vagasLivres == 0) {
84         lcd.print("PARQUE LOTADO");
85         Serial.print('L');
86     } else {
87         lcd.print("BEM-VINDO!");
88         Serial.print('V');
89     }
90 }

```

8.1.1 Análise Lógica do Arduino Gestor

O código do Gestor atua como o cérebro visual do projeto, focando na interface (LEDs e Display LCD) e no controle de estado das vagas. A biblioteca `LiquidCrystal_I2C.h` é utilizada para gerenciar a comunicação com o display de forma simplificada.

No bloco `setup()`, os pinos digitais destinados aos nove LEDs de vagas (2 a 10) e aos LEDs das portas (11 e 12) são configurados como saídas (OUTPUT) e iniciados em estado baixo (LOW).

A função `loop()` funciona de maneira assíncrona, aguardando dados na porta Serial. Seu comportamento segue as seguintes regras de negócio:

- **Entrada:** Ao receber o caractere 'E', o sistema acende o LED indicador da entrada e, caso o limite de vagas não tenha sido atingido (`vagasOcupadas < 9`), incrementa o contador e atualiza o painel. O caractere 'e' apaga o LED indicador.
- **Saída:** Ao receber o caractere 'S', o sistema acende o LED indicador da saída e, desde que o estacionamento não esteja vazio (`vagasOcupadas > 0`), decrementa o contador. O caractere 's' apaga o LED de saída.

Para o controle luminoso, a função `atualizarLuzesEEcra()` realiza uma varredura nos pinos 2 a 10, comparando o índice do pino com a quantidade de vagas ocupadas, acendendo ou apagando os LEDs vermelhos proporcionalmente. Finalmente, a função `atualizarEcra()` calcula as vagas livres (`9 - vagas ocupadas`). Se o valor for zero, exibe "PARQUE LOTADO" e transmite o caractere 'L' via Serial para o Porteiro; caso contrário, exibe "BEM-VINDO!" e transmite 'V' (Vagas disponíveis).

8.2 Código do Arduino Porteiro

Responsável pelo controle dos sensores ultrassônicos e dos servomotores (cancelas).

```
1 #include <Servo.h>
2
3 Servo cancelaEntrada;
4 Servo cancelaSaida;
5
6 // Pinos
7 int pinoSensorEntrada = 3;
8 int pinoMotorEntrada = 5;
9 int pinoSensorSaida = 2;
10 int pinoMotorSaida = 4;
11
12 bool parqueLotado = false;
13 bool carroAtualNaEntrada = false;
14 bool carroAtualNaSaida = false;
15
16 void setup() {
17     Serial.begin(9600);
18     cancelaEntrada.attach(pinoMotorEntrada);
19     cancelaSaida.attach(pinoMotorSaida);
```

```

20  cancelaEntrada.write(0);
21  cancelaSaida.write(0);
22  }
23
24  long lerDistancia(int pino) {
25      pinMode(pino, OUTPUT);
26      digitalWrite(pino, LOW);
27      delayMicroseconds(2);
28      digitalWrite(pino, HIGH);
29      delayMicroseconds(5);
30      digitalWrite(pino, LOW);
31      pinMode(pino, INPUT);
32      return pulseIn(pino, HIGH) / 29 / 2;
33  }
34
35  void loop() {
36      if (Serial.available() > 0) {
37          char mensagemDoGestor = Serial.read();
38          if (mensagemDoGestor == 'L') {
39              parqueLotado = true;
40          } else if (mensagemDoGestor == 'V') {
41              parqueLotado = false;
42          }
43      }
44
45      long distEntrada = lerDistancia(pinoSensorEntrada);
46      long distSaida = lerDistancia(pinoSensorSaida);
47
48      // LÓGICA DA ENTRADA
49      if (distEntrada > 0 && distEntrada < 50) {
50          if (!carroAtualNaEntrada && !parqueLotado) {
51              cancelaEntrada.write(90);
52              Serial.print('E');
53              carroAtualNaEntrada = true;
54          }
55      }
56      else {
57          if (carroAtualNaEntrada) {
58              delay(1000);
59              cancelaEntrada.write(0);
60              Serial.print('e');
61              carroAtualNaEntrada = false;
62          }
63      }
64
65      // LÓGICA DA SAÍDA
66      if (distSaida > 0 && distSaida < 50) {

```

```

67     if (!carroAtualNaSaida) {
68         cancelaSaida.write(90);
69         Serial.print('S');
70         carroAtualNaSaida = true;
71     }
72 }
73 else {
74     if (carroAtualNaSaida) {
75         delay(1000);
76         cancelaSaida.write(0);
77         Serial.print('s');
78         carroAtualNaSaida = false;
79     }
80 }
81
82 delay(100);
83 }

```

8.2.1 Análise Lógica do Arduino Porteiro

Este microcontrolador gerencia o hardware físico de acesso, processando as leituras dos sensores ultrassônicos e acionando as cancelas através da biblioteca `Servo.h`.

A função `lerDistancia()` encapsula a lógica de acionamento do sensor ultrassônico, emitindo um pulso sonoro através do acionamento alternado do pino de saída e convertendo o tempo de retorno do eco (`pulseIn`) em uma distância medida em centímetros.

Dentro do `loop()`, a lógica se divide em duas partes principais:

- **Sincronização de Estado:** Primeiramente, o Porteiro lê os avisos oriundos do Arduino Gestor. Se ler 'L', a variável booleana `parqueLotado` recebe `true`; se ler 'V', recebe `false`. Esta variável é a trava de segurança do sistema.
- **Controle de Acesso (Cancelas):** O sistema mede a distância em ambas as portas constantemente.
 - Se o sensor de entrada detectar um veículo a menos de 50 cm e o estacionamento **não** estiver lotado, o servomotor da entrada desloca-se para 90° (abrindo a cancela) e o caractere 'E' é enviado ao Gestor para iniciar a contagem. Após o veículo passar, um atraso (`delay`) de segurança de 1000 milissegundos é aplicado antes de retornar a cancela a 0° e enviar 'e'.
 - A lógica de saída atua de forma análoga. Se a distância lida for menor que 50 cm, o sistema imediatamente abre a cancela de saída (90°) e notifica o Gestor com a letra 'S', garantindo a liberação do veículo. Após a passagem, a cancela retorna a 0° e o caractere 's' é transmitido.

9 DISTRIBUIÇÃO GERAL DAS RESPONSABILIDADES

Conforme definido ao longo da execução, a distribuição estruturou-se da seguinte maneira:

Tabela 5 – Distribuição de Responsabilidades da Equipe

Integrante	Responsabilidade principal
João Pedro / Deyvison Sousa / Ruan Packrick	Projeto lógico e expressões booleanas
Marcos Antônio / Deyvison Sousa / Ruan Packrick	Implementação do circuito
Leonardo Sampaio / Deyvison Sousa / Ruan Packrick	Testes, ajustes e documentação

Fonte: Elaborada pelos autores (2026).

Nota de transparência: Até a presente fase de execução, o desenvolvimento integral (incluindo o projeto lógico, a modelagem das expressões booleanas, a implementação no Tinkercad e a documentação) foi centralizado no trabalho do integrante Ruan Packrick e Deyvison Sousa.

10 CONSIDERAÇÕES FINAIS

O desenvolvimento do Sistema de Estacionamento Inteligente avançou de forma significativa, com um protótipo funcional no Tinkercad e forte adequação à proposta. O código foi aprimorado para garantir comunicação confiável via porta Serial entre o Arduino Gestor e o Arduino Porteiro, bem como estabilidade na leitura dos sensores ultrassônicos. Apesar das limitações de um ambiente simulado, os resultados evidenciam a aplicação prática bem-sucedida de circuitos digitais, controle lógico e automação na disciplina.

REFERÊNCIAS

IDOETA, I. V.; CAPUANO, F. G. **Elementos de eletrônica digital**. 41. ed. São Paulo: Érica, 2012.

ELECFREAKS. **Ultrasonic Ranging Module HC - SR04**. Datasheet. [S. l.], 2011.

Disponível em:

<<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>>. Acesso em: 07 jun. 2026.

MCROBERTS, M. **Arduino básico**. São Paulo: Novatec Editora, 2011.

MONK, S. **Programação com Arduino**: começando com sketches. Porto Alegre: Bookman, 2013.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. **Sistemas digitais**: princípios e aplicações. 11. ed. São Paulo: Pearson Prentice Hall, 2011.